

Lazy Streams and Infinite Prime Generation in Scheme

Pushpendra

1. Introduction

Streams in Scheme allow representing sequences that are **potentially infinite**. Instead of computing all elements at once, each element is evaluated only when required — an idea known as **lazy evaluation**.

This enables elegant formulations of iterative processes (like Fibonacci, square root improvement, or prime number generation) without loops or explicit state.

2. Delayed Evaluation and Memoization

2.1 Memoized Delays

To avoid recomputation, every delayed expression must store its value once evaluated. This is achieved using the `memo-proc` procedure:

```
(define (memo-proc proc)
  (let ((already-run? false) (result false))
    (lambda ()
      (if (not already-run?)
          (begin (set! result (proc))
                  (set! already-run? true)
                  result)
          result))))
```

Explanation:

- The lambda returned acts as a *promise*.
- When forced for the first time, it executes and stores its result.
- Subsequent calls reuse the stored value — no recomputation.

2.2 Syntax for Delay and Cons-Stream

```
(define-syntax delay
  (syntax-rules ()
    ((delay expr)
     (memo-proc (lambda () expr)))))

(define-syntax cons-stream
  (syntax-rules ()
    ((cons-stream a b)
     (cons a (delay b)))))
```

`delay` wraps expressions in a memoized thunk, while `cons-stream` constructs a stream pair with a delayed tail.

3. Forcing and Accessing Stream Elements

```
(define (force promise) (promise))
(define (stream-car s) (car s))
(define (stream-cdr s) (force (cdr s)))
```

force triggers the evaluation of a promise. Hence, `stream-cdr` evaluates the next part of the stream only when needed.

4. Stream Constructors and Mappers

```
(define the-empty-stream '())
(define (stream-null? s) (eq? s the-empty-stream))

(define (stream-map proc s)
  (if (stream-null? s)
      the-empty-stream
      (cons-stream (proc (stream-car s))
                    (stream-map proc (stream-cdr s)))))

(define (stream-filter pred s)
  (cond ((stream-null? s) the-empty-stream)
        ((pred (stream-car s))
         (cons-stream (stream-car s)
                       (stream-filter pred (stream-cdr s))))
        (else (stream-filter pred (stream-cdr s)))))
```

These definitions allow functional operations (mapping, filtering) over infinite sequences.

5. Infinite Numeric Streams

5.1 Infinite Integers

```
(define (integers-starting-from n)
  (cons-stream n (integers-starting-from (+ n 1))))
(define integers (integers-starting-from 1))
```

This produces an infinite sequence of integers lazily.

5.2 Fibonacci Sequence

```
(define (fibgen a b)
  (cons-stream a (fibgen b (+ a b))))
(define fibs (fibgen 0 1))
```

Each Fibonacci term is computed only when requested and reuses previous terms via memoization.

6. Prime Number Generation Using Streams

6.1 Primality Test (Direct)

```
(define (square x) (* x x))

(define (prime? n)
  (define (divides? a b) (= (remainder b a) 0))
  (define (smallest-divisor n) (find-divisor n 2))
  (define (find-divisor n test-divisor)
    (cond ((> (square test-divisor) n) n)
          ((divides? test-divisor n) test-divisor)
          (else (find-divisor n (+ test-divisor 1)))))
  (= n (smallest-divisor n)))
```

This checks if a number has any divisors up to its square root.

6.2 Stream-Based Sieve of Eratosthenes

```
(define (divisible? x y) (= (remainder x y) 0))

(define (sieve s)
  (cons-stream
    (stream-car s)
    (sieve (stream-filter
      (lambda (x)
        (not (divisible? x (stream-car s))))
      (stream-cdr s)))))

(define primes (sieve (integers-starting-from 2)))
```

How It Works:

- Start with the stream of integers from 2.
- The head of the stream (`stream-car`) is the next prime.
- The tail is filtered lazily to remove multiples of that prime.
- The process repeats recursively, generating primes infinitely.

Because of lazy evaluation, only as many primes as needed are generated — the stream expands on demand.

7. Alternative Prime Generation (Using Previously Found Primes)

```
(define primes2
  (cons-stream 2
    (stream-filter prime2? (integers-starting-from 3))))

(define (prime2? n)
  (define (iter ps)
    (cond ((> (square (stream-car ps)) n) true)
          ((divisible? n (stream-car ps)) false)
          (else (iter (stream-cdr ps)))))
  (iter primes2))
```

Here, `prime2?` uses the already generated primes stream `primes2` to test new numbers — a beautiful self-referential definition enabled by laziness.

8. Stream Reuse: Square Root Example

```
(define (average x y) (/ (+ x y) 2))
(define (sqrt-improve guess x)
  (average guess (/ x guess)))

(define (sqrt-stream x)
  (define guesses
    (cons-stream 1.0
      (stream-map (lambda (guess)
        (sqrt-improve guess x))
        guesses)))
  guesses)
```

Key Idea: The stream `guesses` refers to itself — each new approximation uses the previous guess stored in the same stream.

9. Summary

- Streams implement lazy lists — elements are computed only when needed.
- Memoization ensures previously computed values are reused.
- The sieve and Fibonacci examples demonstrate recursive streams that depend on earlier computed elements.
- Infinite structures are handled naturally via delayed evaluation.